CLIP LINUX

Multi-Distribution Architecture Addendum

Companion document to the Ubuntu Clip Implementation Blueprint · v1.0

Purpose

Extend the original Ubuntu-focused plan into a Linux desktop application that can run across distributions and desktop environments without duplicating its core clipboard engine.

> **Core principle**
> One shared clipboard engine. Multiple desktop adapters. Multiple package formats.

1. Product direction

Keep the product name and codebase distribution-neutral: **Clip** or **Clip Linux**. Ubuntu Yaru remains the initial visual theme, but theme tokens must be replaceable and the UI should follow the host desktop's light/dark preference where possible.

The shared Rust core must not depend on GNOME Shell, KDE Plasma, Cinnamon, XFCE, or a distribution-specific package manager. Distribution-specific behavior belongs in adapters and packaging.

2. Support strategy and realistic compatibility

Target	Desktop/session	Implementation approach	Initial status
Ubuntu	GNOME · Wayland/X11	First reference integration; GNOME panel/shortcut adapter	Primary development target
Fedora	GNOME · Wayland	Reuse GNOME adapter; validate portal and clipboard behavior	Next validation target
Debian	GNOME · Wayland/X11	Reuse GNOME adapter; package dependencies may differ	Next validation target
Arch Linux	GNOME / KDE	Reuse desktop adapters; provide PKGBUILD/AUR recipe	Next validation target
Linux Mint	Cinnamon · X11/Wayland where available	Cinnamon panel/shortcut adapter; test session support	Later adapter
KDE Neon / openSUSE	KDE Plasma · Wayland/X11	KDE integration and shortcut/clipboard adapter	Later adapter
XFCE-based distros	XFCE · primarily X11	Panel plugin or tray fallback; test global shortcuts	Later adapter
COSMIC and others	Varies	Use generic UI and available desktop portals; feature-detect capabilities	Community/experimental

Important: this is a support roadmap, not a claim that all combinations already work. Wayland clipboard-manager access and automated paste are compositor/protocol dependent. Maintain a tested matrix by distribution, desktop version, session type, and feature.

3. Revised cross-desktop architecture

Keep these layers separate:

Shared product core (Rust)
Capture model · normalization · deduplication · search · SQLite · blob store · backup · restore · retention

Platform capability API
Clipboard read/write · change notifications · global shortcut · panel/menu entry · notifications · paste request · autostart

Desktop adapters
GNOME adapter · KDE adapter · Cinnamon adapter · XFCE adapter · generic portal/tray fallback

Presentation and packaging
GTK4/Libadwaita UI · theme tokens · Flatpak · AppImage · DEB · RPM · Arch PKGBUILD

Define Rust traits/interfaces so the core depends on capabilities, not desktop names. Example:

```
trait ClipboardBackend {
fn capabilities(&self) -> ClipboardCapabilities;
async fn read_available(&self) -> Result<ClipboardPayload>;
async fn write(&self, payload: ClipboardPayload) -> Result<()>;
fn subscribe_changes(&self) -> ChangeStream;
}

trait DesktopIntegration {
fn capabilities(&self) -> DesktopCapabilities;
```

```
async fn request_paste(&self) -> Result<PasteOutcome>;
async fn show_panel_entry(&self) -> Result<()>;
async fn register_shortcut(&self, shortcut: Shortcut) -> Result<()>;
}
```

The exact async/runtime and object-safety design should be decided during implementation. The important contract is capability detection and graceful absence: unsupported features return an explicit Unsupported result rather than silently failing.

4. Desktop integration and Wayland strategy

Use desktop portals and standard protocols where they provide the needed capability. Do not assume a portal exposes arbitrary clipboard history or synthetic paste on every desktop. Keep compositor-specific integration behind adapters and document feature differences.

Clipboard capture: prove the required clipboard-manager mechanism on each supported Wayland compositor. On X11, use an X11-specific backend and test clipboard ownership/selection behavior. Support MIME types and ownership lifetimes correctly.

Global shortcut: prefer the XDG Desktop Portal GlobalShortcuts interface where supported and user-authorized. A GNOME Shell or KDE-specific shortcut adapter can be a fallback. Detect conflicts and let users configure another shortcut.

Paste: restoring content to the clipboard is the reliable baseline. Automated paste is an optional capability and must be tested per desktop/session. Always retain copy-to-clipboard plus manual Ctrl+V fallback.

5. UI and theming across desktops

Keep the compact popup and keyboard-first interaction consistent. Use GTK4 + Libadwaita for the first UI, but isolate styling behind design tokens: background, elevated surface, text, muted text, accent, focus, border, radius, spacing and typography.

Default theme can be Ubuntu Yaru-inspired aubergine/orange. On non-Ubuntu systems, offer System, Yaru Dark, Yaru Light and Neutral options. Follow system light/dark preference when available; do not assume the host has Yaru installed.

Panel integration should be native where practical, but a tray/launcher fallback is acceptable on desktops where a custom panel extension is not available. The core popup should also be launchable independently.

6. Packaging and release strategy

Format	Target	Notes
Flatpak	Many desktop distributions	Good broad distribution path; validate clipboard, portal, autostart and background-service permissions. Flatpak is not a magic fix for every desktop integration.
DEB	Ubuntu, Debian, Mint	Use distro-compatible dependencies and maintain package upgrade scripts.
RPM	Fedora, openSUSE	Build and test against supported release versions.
Arch PKGBUILD/AUR	Arch, Manjaro	Provide reproducible build instructions; community-maintained AUR recipe if appropriate.
AppImage	Portable fallback	Useful for UI distribution, but global shortcuts, portals, autostart and daemon integration still need host setup.

Recommendation: ship native packages for the first supported distros, then Flatpak once permissions and portal behavior are verified. Keep daemon installation and UI packaging explicit; avoid a package that appears installed but cannot capture clipboard changes due to missing session integration.

7. Suggested repository structure

```
clip-linux/
apps/
gtk-ui/
services/
clipboard-daemon/
crates/
domain/
clipboard-core/
storage-sqlite/
blob-store/
```

```
search/
backup/
recovery/
ipc-dbus/
platform-api/
integrations/
gnome/
kde/
cinnamon/
xfce/
generic/
packaging/
flatpak/
deb/
rpm/
arch/
appimage/
tests/
core/
crash-recovery/
backup-restore/
platform-matrix/
docs/
architecture/
compatibility/
packaging/
```

8. Revised implementation roadmap

Stage 0 — Portability spike

Pick one Ubuntu GNOME target and one secondary desktop (recommend Fedora GNOME or Arch KDE). Prove clipboard capture, restore, shortcut, and paste capabilities before building abstractions.

Stage 1 — Portable core

Implement Rust domain model, SQLite/blob storage, search, backup/restore and tests with no desktop-specific dependencies.

Stage 2 — Capability API

Define platform interfaces and explicit capability reporting. Add fake/test backends so core tests do not require a desktop session.

Stage 3 — First adapter

Complete GNOME integration and GTK UI on the reference Ubuntu release.

Stage 4 — Second desktop proof

Implement KDE Plasma adapter or validate generic portal support. Record exact feature gaps rather than forcing parity.

Stage 5 — Packaging

Produce DEB and RPM first; add Arch PKGBUILD and Flatpak after permission and lifecycle tests.

Stage 6 — Expand support

Add Cinnamon, XFCE and other desktops based on user demand and test capacity.

9. Multi-distro release acceptance checklist

- ■ Core unit and storage tests pass without a running desktop.
- ■ Clipboard capture and restore tested on each advertised desktop/session.
- ■ Global shortcut registration tested, including conflict handling.
- ■ Automated paste is labeled supported only where it passed real application tests.
- ■ System theme and Yaru theme both render correctly.
- ■ Service starts and restarts correctly under the target user session manager.
- ■ Upgrade and uninstall preserve user data unless the user explicitly chooses removal.
- ■ Backup and restore work across supported package upgrades and schema migrations.
- ■ Package dependencies and file paths follow distro conventions.

■ Compatibility page lists distro version, desktop version, session, and supported capabilities.

10. Technical references

• **XDG Desktop Portal documentation** — https://flatpak.github.io/xdg-desktop-portal/

• **Global Shortcuts portal interface** — https://flatpak.github.io/xdg-desktop-portal/docs/doc-org.freedesktop.portal.GlobalShortcuts.html

• **Wayland protocols** — https://wayland.app/protocols/

• **Flatpak documentation** — https://docs.flatpak.org/

• **GNOME Shell extensions** — https://gjs.guide/extensions/

• **KDE developer documentation** — https://develop.kde.org/

• **SQLite WAL** — https://www.sqlite.org/wal.html

• **SQLite Online Backup API** — https://www.sqlite.org/backup.html

Final recommendation: Keep the original Ubuntu Clip blueprint as the product and reliability specification. Use this addendum to make the codebase distribution-neutral. Begin with one GNOME target, prove a second desktop early, and expand only when the capability matrix and automated tests support the claim.